

## Ratings encoding

In the table below, each row represents a user's ratings of movies: ✓ (check) indicates the person liked the movie, ✗ (x) that they didn't, and • (dot) that they didn't rate it one way or another (neutral rating or didn't watch). Can encode these ratings numerically with 1 for ✓ (check), −1 for ✗ (x), and 0 for • (dot).

| Person | Superman | Wicked | KPop Demon Hunters | A Minecraft Movie | Ratings written as a 4-tuple |
|--------|----------|--------|--------------------|-------------------|------------------------------|
| $P_1$  | ✗        | •      |                    | ✓                 |                              |
| $P_2$  | ✓        | ✓      |                    | ✗                 |                              |
| $P_3$  | ✓        | ✓      |                    | ✓                 |                              |
| $P_4$  | •        | ✗      |                    | ✓                 |                              |
| $You$  |          |        |                    |                   |                              |

## Defining sets

*To define sets:*

To define a set using **roster method**, explicitly list its elements. That is, start with { then list elements of the set separated by commas and close with }.

To define a set using **set builder definition**, either form “The set of all  $x$  from the universe  $U$  such that  $x$  is ...” by writing

$$\{x \in U \mid ...x...\}$$

or form “the collection of all outputs of some operation when the input ranges over the universe  $U$ ” by writing

$$\{...x... \mid x \in U\}$$

We use the symbol  $\in$  as “is an element of” to indicate membership in a set.

**Example sets:** For each of the following, identify whether it's defined using the roster method or set builder notation and give an example element.

Can we infer the data type of the example element from the notation?

$$\{-1, 1\}$$

$$\{0, 0\}$$

$$\{-1, 0, 1\}$$

$$\{(x, x, x) \mid x \in \{-1, 0, 1\}\}$$

$$\{\}$$

$$\{x \in \mathbb{Z} \mid x \geq 0\}$$

$$\{x \in \mathbb{Z} \mid x > 0\}$$

$$\{\smile, \odot\}$$

$$\{\text{A, C, U, G}\}$$

$$\{\text{AUG, UAG, UGA, UAA}\}$$

## Defining functions ratings

Recall our representation of Netflix users' ratings of movies as  $n$ -tuples, where  $n$  is the number of movies in the database. Each component of the  $n$ -tuple is  $-1$  (didn't like the movie),  $0$  (neutral rating or didn't watch the movie), or  $1$  (liked the movie).

Consider the ratings  $P_1 = (-1, 0, 1, 0)$ ,  $P_2 = (1, 1, -1, 0)$ ,  $P_3 = (1, 1, 1, 0)$ ,  $P_4 = (0, -1, 1, 0)$

Which of  $P_1$ ,  $P_2$ ,  $P_3$  has movie preferences most similar to  $P_4$ ?

One approach to answer this question: use **functions** to quantify difference among user preferences.

For example, consider the function  $d_0 : \{-1, 0, 1\}^4 \times \{-1, 0, 1\}^4 \rightarrow \mathbb{R}$  given by

$$d_0((x_1, x_2, x_3, x_4), (y_1, y_2, y_3, y_4)) = \sqrt{(x_1 - y_1)^2 + (x_2 - y_2)^2 + (x_3 - y_3)^2 + (x_4 - y_4)^2}$$

## Defining functions more examples

Let's practice with functions related to some of our applications so far.

Recall: We model the collection of user ratings of the four movies Superman, Wicked, KPop Demon Hunters, A Minecraft Movie as the set  $\{-1, 0, 1\}^4$ . One function that compares pairs of ratings is

$$d_0 : \{-1, 0, 1\}^4 \times \{-1, 0, 1\}^4 \rightarrow \mathbb{R}$$

given by

$$d_0((x_1, x_2, x_3, x_4), (y_1, y_2, y_3, y_4)) = \sqrt{(x_1 - y_1)^2 + (x_2 - y_2)^2 + (x_3 - y_3)^2 + (x_4 - y_4)^2}$$

Notice: any ordered pair of ratings is an okay input to  $d_0$ .

Notice: there are (at most)

$$(3 \cdot 3 \cdot 3 \cdot 3) \cdot (3 \cdot 3 \cdot 3 \cdot 3) = 3^8 = 6561$$

many pairs of ratings. There are therefore lots and lots of real numbers that are not the output of  $d_0$ .

Recall: RNA is made up of strands of four different bases that encode genomic information in specific ways. The bases are elements of the set  $B = \{\mathbf{A}, \mathbf{C}, \mathbf{U}, \mathbf{G}\}$ . The set of RNA strands  $S$  is defined (recursively) by:

Basis Step:  $\mathbf{A} \in S, \mathbf{C} \in S, \mathbf{U} \in S, \mathbf{G} \in S$

Recursive Step: If  $s \in S$  and  $b \in B$ , then  $sb \in S$

where  $sb$  is string concatenation.

**Pro-tip:** informal definitions sometime use  $\dots$  to indicate “continue the pattern”. Often, to make this pattern precise we use recursive definitions.

| Name          | Domain | Codomain       | Rule                                                                                                                                                                                                          | Example                                                                                                                |
|---------------|--------|----------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------|
| <i>rnalen</i> | $S$    | $\mathbb{Z}^+$ | <p>Basis Step:<br/> If <math>b \in B</math> then <math>rnalen(b) = 1</math><br/> Recursive Step:<br/> If <math>s \in S</math> and <math>b \in B</math>, then<br/> <math>rnalen(sb) = 1 + rnalen(s)</math></p> | $rnalen(\mathbf{AC}) \stackrel{\text{rec step}}{=} 1 + rnalen(\mathbf{A})$ $\stackrel{\text{basis step}}{=} 1 + 1 = 2$ |

|                  |              |              |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                |                                             |
|------------------|--------------|--------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------|
| <i>basecount</i> | $S \times B$ | $\mathbb{N}$ | <p>Basis Step:<br/> If <math>b_1 \in B, b_2 \in B</math> then<br/> <math>basecount( (b_1, b_2) ) =</math><br/> <math display="block">\begin{cases} 1 &amp; \text{when } b_1 = b_2 \\ 0 &amp; \text{when } b_1 \neq b_2 \end{cases}</math><br/> Recursive Step:<br/> If <math>s \in S, b_1 \in B, b_2 \in B</math><br/> <math>basecount( (sb_1, b_2) ) =</math><br/> <math display="block">\begin{cases} 1 + basecount( (s, b_2) ) &amp; \text{when } b_1 = b_2 \\ basecount( (s, b_2) ) &amp; \text{when } b_1 \neq b_2 \end{cases}</math></p> | $basecount( (\mathbf{ACU}, \mathbf{C}) ) =$ |
|------------------|--------------|--------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------|

|                     |              |              |                                                                                                                    |  |
|---------------------|--------------|--------------|--------------------------------------------------------------------------------------------------------------------|--|
| “2 to the power of” | $\mathbb{N}$ | $\mathbb{N}$ | <p>Basis Step:<br/> <math>2^0 = 1</math><br/> Recursive Step:<br/> If <math>n \in \mathbb{N}, 2^{n+1} =</math></p> |  |
|---------------------|--------------|--------------|--------------------------------------------------------------------------------------------------------------------|--|

|                             |                                  |              |                                                                                                                                |  |
|-----------------------------|----------------------------------|--------------|--------------------------------------------------------------------------------------------------------------------------------|--|
| “ $b$ to the power of $i$ ” | $\mathbb{Z}^+ \times \mathbb{N}$ | $\mathbb{N}$ | <p>Basis Step:<br/> <math>b^0 = 1</math><br/> Recursive Step:<br/> If <math>i \in \mathbb{N}, b^{i+1} = b \cdot b^i</math></p> |  |
|-----------------------------|----------------------------------|--------------|--------------------------------------------------------------------------------------------------------------------------------|--|

|           |           |           |           |            |            |            |             |             |             |                 |
|-----------|-----------|-----------|-----------|------------|------------|------------|-------------|-------------|-------------|-----------------|
| $2^0 = 1$ | $2^1 = 2$ | $2^2 = 4$ | $2^3 = 8$ | $2^4 = 16$ | $2^5 = 32$ | $2^6 = 64$ | $2^7 = 128$ | $2^8 = 256$ | $2^9 = 512$ | $2^{10} = 1024$ |
|-----------|-----------|-----------|-----------|------------|------------|------------|-------------|-------------|-------------|-----------------|